Reliable File Transfer over UDP with SDN Path Control

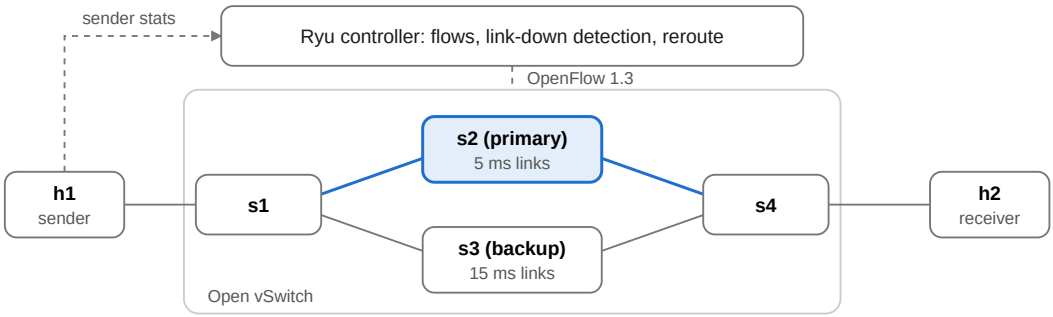
CN Mini Project (SDN–Socket Programming, problem 4) · D1 design document · 27 Oct 2026

1. Problem understanding and architecture

UDP gives no delivery, ordering or duplicate protection, so we build a reliable file-transfer protocol on UDP sockets and run it over a Mininet network with two disjoint paths. An SDN controller forwards traffic on a primary path and moves it to a backup path when a link fails.

- **Sender (h1) and receiver (h2):** Python UDP sockets; all reliability logic lives in the application (`src/`).
- **Topology:** h1 – s1 – {s2 or s3} – s4 – h2. Primary path s1-s2-s4 (5 ms links), backup s1-s3-s4 (15 ms links), so a reroute is visible in the RTT.
- **Controller:** a Ryu OpenFlow 1.3 app that installs forwarding flows, detects link-down events and reroutes (`controller/`).
- **Feedback channel:** the sender reports retransmissions and RTT to the controller (section 4).

The diamond contains a loop, so ARP is static and the switch table-miss rule drops; no broadcast can circulate.



Blue = active path (s1-s2-s4) until a link on it fails. Solid = data links, dashed = control and feedback.

2. UDP socket implementation

Plain `SOCK_DGRAM` sockets with Selective Repeat on top. Each datagram has a 12-byte header (magic, type, session, seq, length), the payload, and a CRC-32 trailer.

Requirement	Mechanism	File
Sequence numbering	File cut into 1024-byte chunks; chunk *i* carries `seq = i`.	`protocol.py`
Acknowledgments	Receiver ACKs every DATA packet individually, duplicates included, because the first ACK may have been lost.	`receiver.py`
Timeout and retransmission	Window of 32 chunks, one timer per chunk, only the expired chunk is resent. RTO follows RFC 6298 (SRTT, RTTVAR), limited to 50 ms–2 s, doubles per retry, no RTT sample from retransmitted packets (Karn's rule).	`sender.py`
Missing and duplicate packets	Chunk written at offset seq × 1024. A seq already seen is a duplicate; a seq above highest + 1 is a gap; a seq below the highest is out of order. Each is counted.	`receiver.py`
File integrity	CRC-32 on every packet (corrupt packets are dropped and recovered by timeout). The file's SHA-256 travels in FIN; the receiver compares it with the reassembled file and answers in FIN_ACK.	both
Lost control packets	START and FIN are retried with back-off.	`sender.py`

Design decisions. Selective Repeat over Go-Back-N: one loss costs one retransmission and gaps and duplicates stay visible at the receiver. Per-chunk ACKs instead of cumulative: a lost ACK is harmless. A 1024-byte chunk keeps each datagram under the 1500-byte MTU, so there is no IP fragmentation.

3. Mininet and SDN implementation

`topology/diamond_topo.py` builds four Open vSwitch switches and two hosts with `TCLink` (10 Mb/s; 5 ms on the primary path, 15 ms on the backup). Ports are numbered explicitly so the controller can hard-code them: s1 (1 = h1, 2 = s2, 3 = s3), s2 and s3 (1 = s1, 2 = s4), s4 (1 = h2, 2 = s2, 3 = s3). `controller/sdn_controller.py` is a Ryu OpenFlow 1.3 app:

- **Proactive forwarding:** when a switch connects, flows matching IPv4 `ipv4_dst` (priority 100) output on the active path's ports. The table-miss rule drops.
- **Failure detection:** a `PortStatus` event with link-down or port-down marks that (switch, port) as down.
- **Reroute:** if a down port belongs to the active path and the other path is healthy, the controller reinstalls flows for the other path. Match and priority are identical, so the new actions replace the old ones in place.
- **Failure injection:** `--fail-at N` runs `configLinkStatus s1 s2 down` N seconds into the transfer.

D1 limits: reroute happens only on link-down (not on degradation), and there is no fail-back to the primary path.

4. Socket–SDN integration

1. **Path control:** the controller decides which path the sender's UDP packets take. When it reroutes, the RTT changes and packets lost during the switchover are recovered by the sender's timeouts and retransmissions, so the application needs no change.
2. **Explicit feedback:** about once a second, and again at completion, the sender sends the controller a JSON datagram: chunks acked and total, retransmissions, timeouts, SRTT and RTO. The controller keeps the latest values and logs them.

The feedback goes over a Unix datagram socket (`/tmp/sdn_app_stats.sock`): Mininet hosts have their own network namespace, so `127.0.0.1` inside h1 does not reach the controller, but a socket file is shared across namespaces. A plain UDP address (`--report HOST:PORT`) is also supported. In D1 the stats are only logged. D2 will use retransmission rate and SRTT to detect a degraded path without a link failure, reroute, and compare against a static-flow baseline.

5. Initial testing

All 10 protocol scenarios pass: the received file is byte-identical and its SHA-256 is verified. `tests/impair_proxy.py` sits between sender and receiver on localhost and drops, duplicates, reorders or corrupts packets in both directions (500 KB file unless noted).

Scenario	Time (s)	Retransmissions	Duplicates seen	Corrupt dropped
Clean	0.08	0	0	0
10% loss	3.07	121	59	0
25% loss (300 KB)	13.87	222	85	0
20% duplicated	0.05	0	92	0
30% reordered	0.63	4	4	0
10% corrupted	5.22	131	56	75
Loss + duplication + reordering + corruption	6.04	201	141	35

`tests/test_report_channel.py` receives the sender's reports on a Unix socket and checks that the last one shows all chunks acked. *[Mininet and Ryu run: add the controller log lines `PORT DOWN` and `REROUTE: primary -> backup` and the sender's final integrity result here.]*

6. How to build and run

Target: Ubuntu 22.04, Python 3.10. Run every command from the repository root.

```
# 1. Install once (enable the 'universe' repository first if apt cannot find mininet)
sudo apt install -y mininet openvswitch-switch python3-venv
python3 -m venv ~/ryu-venv && source ~/ryu-venv/bin/activate
pip install wheel "setuptools==67.6.1" && pip install "eventlet==0.30.2" ryu
deactivate
# 2. Protocol tests (no root, no Mininet)
python3 tests/run_local_tests.py && python3 tests/test_report_channel.py
# 3. Terminal 1: controller
source ~/ryu-venv/bin/activate && ryu-manager controller/sdn_controller.py
# 4. Terminal 2: normal transfer, then link-failure transfer
sudo mn -c && sudo python3 topology/diamond_topo.py --transfer --size-mb 5 --tag normal
sudo mn -c && sudo python3 topology/diamond_topo.py --transfer --size-mb 20 --fail-at 2 --tag failover
```

Expected: step 2 ends with `ALL PASSED` and `RESULT: PASS`. In step 4 the controller prints `switch s1 connected` to `s4 connected`, the sender ends with `integrity VERIFIED`, and the failover run also prints `PORT DOWN` and `REROUTE: primary -> backup`. Result files are written to `results/`.